z0p — Yellow Paper

Formal Specification of the z0p Proving Stack

nul0
Future Proofs Tech
futureproofs@proton.me

November 2026

Abstract

This document specifies the mathematics, protocols, encodings, parameter validation, and security invariants implemented by **z0p** version 0.3.0. z0p is a from-scratch, zero-dependency zero-knowledge proving stack in safe Rust: binary fields, the BabyBear prime field, a quartic extension field, BLAKE3-256, Merkle trees, a parallel NTT, FRI, sumcheck, univariate and multilinear polynomial commitments, LogUp lookups, STARK-style AIR systems, hash-folded IVC, a register-machine AIR, and zero-knowledge masking. Normative requirements are stated explicitly. Conjectured bit-levels are marked as such. The production recipe targets a conjectured 100-bit query/proof-of-work schedule (rate 1/16, 40 queries, 20-bit grinding); it is not an end-to-end soundness theorem. The artifact is unaudited research software.

Contents

1 Conventions and notation **2**

2 Fields **2**
2.1 BabyBear . . . 2
2.2 Quartic extension . . . 3
2.3 Binary fields . . . 4

3 Domains, NTT, and Reed–Solomon encoding **4**

4 BLAKE3 and Merkle commitments **5**

5 Fiat–Shamir transcript and proof of work **5**

6 FRI polynomial commitment **6**
6.1 Parameters and commitment . . . 6
6.2 Arity-4 folding . . . 7
6.3 Proof and verification . . . 7
6.4 Conjectured soundness . . . 7

7 Multilinear polynomials and sumcheck **8**

8 Univariate, Gemini, and BaseFold commitments **8**
8.1 UPCs . . . 8
8.2 Gemini . . . 8
8.3 BaseFold prototype . . . 8

9 Lookup arguments **9**
9.1 LogUp in clear . . . 9
9.2 Committed LogUp . . . 9

10 AIR systems **9**
10.1 Fibonacci AIR . . . 9
10.2 Range check AIR . . . 9
10.3 Table membership AIR . . . 10
10.4 Hash-folded IVC . . . 10
10.5 Register-machine execution . . . 10
10.6 Recursion fragment . . . 10

11 Zero-knowledge masking **10**

12 Wire format **11**

13 Parameter validation and sizing **12**

14 Security analysis **12**

15 Evidence and reproducibility **13**

A Error taxonomy **14**

B Public API map **14**

C Parameter quick reference **15**

D Glossary **15**

E Reproducibility checklist **15**

1 Conventions and notation

Normative language. **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are normative. A proof object is the Rust value; its encoding is the canonical byte string produced by `to_bytes`. A verifier MUST reject on any failed check with `Err` and MUST NOT panic on untrusted input. Public `Params` structs have public fields; every entry point MUST re-validate them. Integer encodings are little-endian. `usize` is the platform word.

2 Fields

2.1 BabyBear

$$p = 2^{31} - 2^{27} + 1 = 2013265921, \qquad p - 1 = 2^{27} \cdot 15, \qquad g = 31.$$

Elements are canonical integers in $[0, p)$. The private representation is a `u32` with invariant `value < p`. The public constructor reduces modulo p; the decoder rejects non-canonical bytes.

Addition and subtraction. For $a, b < p$:

$$a + b = \begin{cases} a + b, & a + b < p, \\ a + b - p, & \text{otherwise}, \end{cases} \qquad a - b = \begin{cases} a - b, & a \geq b, \\ a - b + p, & \text{otherwise}. \end{cases}$$

Both fit in `u32` because $2p < 2^{32}$. Negation maps $0 \mapsto 0$ and $a \mapsto p - a$.

Symbol	Meaning
p	BabyBear modulus $2^{31} - 2^{27} + 1 = 2013265921$
$\mathbb{F}_p$	$\mathbb{Z}/p\mathbb{Z}$
$\mathbb{F}_{p^2}$	$\mathbb{F}_p[u]/(u^2 - 11)$
$\mathbb{F}_{p^4}$	$\mathbb{F}_{p^2}[v]/(v^2 - u)$
H	base trace domain of size N
D	coset LDE domain of size $M = N\beta$
β	blowup factor, power of two
ω, Ω	primitive N-th and M-th roots of unity
γ	coset shift `LDE_SHIFT`=31
$Z_H(x)$	$x^N - 1$
w_{last}	ω^{N-1}
Q	FRI query count
f	FRI fold count
F	final layer size
$H(x)$	BLAKE3-256 digest

Table 1: Notation used throughout the specification.

Multiplication. Let

$$\text{MU} = \lfloor 2^{64}/p \rfloor = 9162596893, \quad x = ab < p^2 < 2^{62}, \quad q = ((x \gg 32) \cdot \text{MU}) \gg 32.$$

Then $q \le \lfloor x/p \rfloor$ and $r = x - qp < 2p$; a short loop subtracts p until $r < p$. This Barrett reduction is cross-checked against wide `%` on 100,000 random pairs and edge values.

Inversion and halving. Inversion is Fermat $a^{-1} = a^{p-2}$, with $0^{-1} = 0$ by convention. `halve(a)` computes $(a + (a \bmod 2)p)/2$, exact because p is odd. Batch inversion uses Montgomery's trick: one field inversion and $3n$ multiplications; zeros map to zero and do not contribute to the running product.

Roots of unity. For $k \le 27$, `two_adic_generator`$(k) = g^{(p-1)/2^k}$ generates the unique subgroup of order 2^k. `primitive_root`(m) requires a power of two $m \le 2^{27}$.

2.2 Quartic extension

Define

$$\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11), \qquad \mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u).$$

An element is $a + bu + cv + duv$, stored as four canonical $\mathbb{F}_p$ coordinates.

Multiplication. Write $x = A + Cv$, $y = B + Dv$ with $A, B, C, D \in \mathbb{F}_{p^2}$. Then

$$xy = (AB + uCD) + (AD + CB)v,$$

using Karatsuba for the cross term $\text{cross} = (A + C)(B + D) - AB - CD$. $\mathbb{F}_{p^2}$ multiplication is

$$(a_0 + a_1 u)(b_0 + b_1 u) = (a_0 b_0 + 11 a_1 b_1) + (a_0 b_1 + a_1 b_0)u.$$

Inversion. For $x = c + dv$ with $c, d \in \mathbb{F}_{p^2}$,

$$x^{-1} = (c - dv)/(c^2 - ud^2).$$

The denominator inverse uses the $\mathbb{F}_{p^2}$ norm

$$(c_0 + c_1 u)^{-1} = (c_0 - c_1 u)/(c_0^2 - 11 c_1^2).$$

One $\mathbb{F}_{p^2}$ inversion therefore suffices. `inv(0)=0`; no zero branch is used in the field core.

Canonical encoding. Each coordinate is four canonical little-endian bytes; an element is 16 bytes. The decoder checks every coordinate against p.

2.3 Binary fields

Field	Polynomial
$GF(2^8)$	$x^8 + x^4 + x^3 + x + 1$ (`0x11B`)
$GF(2^{64})$	$x^{64} + x^4 + x^3 + x + 1$
$GF(2^{128})$	$x^{128} + x^7 + x^2 + x + 1$

Table 2: Binary fields used for program words.

Addition is XOR. Multiplication is carryless multiplication followed by reduction modulo the field polynomial. Arithmetic loops run fixed iteration counts; inverses use fixed Fermat exponentiation and map zero to zero. Every bit pattern is a valid element. `F128::embed_f64` is not provided: a bit-level embedding of `F64` into `F128` is not a field homomorphism because the `F64` modulus does not divide the `F128` modulus.

3 Domains, NTT, and Reed–Solomon encoding

Domains. The base domain is

$$H = \{1, \omega, \ldots, \omega^{N-1}\}, \qquad \omega = \texttt{primitive_root}(N).$$

The evaluation domain is the coset

$$D = \gamma \cdot \langle \Omega \rangle, \qquad |D| = M = N\beta,$$

with $\Omega = \texttt{primitive_root}(M)$ and $\gamma = 31$. The ceiling is $M \le 2^{27}$. The vanishing polynomial of H is $Z_H(x) = x^N - 1$.

Bit reversal. For $n = 2^k$, index i maps to

$$\mathrm{rev}_k(i) = \mathrm{reverse_bits}(i) \gg (\texttt{usize::BITS} - k).$$

Small arrays are permuted in place; large arrays gather into a fresh buffer in parallel. FRI layers are bit-reversed so each adjacent quad is $(x, -x, j_0 x, -j_0 x)$.

NTT. The forward transform evaluates coefficients at powers of ω; the inverse interpolates and scales by $1/n$. Cooley–Tukey stages:

$$\begin{aligned} u &= \mathrm{block}[j], \\ v &= \mathrm{block}[j + \ell/2] \cdot \mathrm{twiddle}[j \cdot (n/\ell)], \\ \mathrm{block}[j] &= u + v, \\ \mathrm{block}[j + \ell/2] &= u - v. \end{aligned}$$

Twiddles are chained powers $1, \mathrm{root}, \ldots, \mathrm{root}^{n/2-1}$. The inverse folds $1/n$ into the final stage. Domains must be nonempty powers of two at most 2^{27}.

Coset evaluation. For coefficients $c_0, \ldots, c_{n-1}$ and blowup β, compute $M = n\beta$ values $c(\gamma\Omega^i)$ by multiplying c_i by γ^i, zero-padding to M, and applying the forward NTT. Extension coset evaluation applies the base transform to each of the four coordinates and recombines.

4 BLAKE3 and Merkle commitments

BLAKE3-256. Unkeyed BLAKE3 with 256-bit output: 16-word state, 7 ARX rounds, message permutation $[2, 6, 3, 10, 7, 0, 4, 13, 1, 11, 12, 5, 9, 14, 15, 8]$, flags `CHUNK_START=1`, `CHUNK_END=2`, `PARENT=4`, `ROOT=8`, 1024-byte chunks, 64-byte blocks, and Bao-style merge-while-even stacking. Validated against reference vectors for empty input, `abc`, one chunk, one byte over a chunk, two chunks, and multi-chunk inputs crossing boundaries. Incremental updates with arbitrary splits equal the one-shot digest.

Merkle trees. A complete binary tree over leaves padded by duplicating the last leaf to a power of two. For raw leaves:

$$\text{leaf}(d) = H(0x00\|d), \qquad \text{node}(l, r) = H(0x01\|l\|r).$$

FRI quad trees and AIR trace trees use 64-byte leaves hashed with standard BLAKE3 (one compression per full block); internal nodes use the same `0x01` construction.

A path is a sequence (sibling, is_left). Verification recomputes the leaf digest, folds upward, and compares to the root. A path is valid for index i only if for every level k,

$$\text{is_left}_k = ((i \gg k)\&1 = 0).$$

This index binding is mandatory. Without it, a valid opening for leaf j would verify at any other in-range index with the same path length.

Caps. A cap at drop d is

$$\text{cap}(d) = \text{layers}[\text{depth} - \min(d, \text{depth})],$$

containing $2^{\min(d,\text{depth})}$ digests. Openings omit the top d path steps and authenticate the remaining path to the selected cap entry. Cap height is capped at 8 in every public `Params`; `MAX_CAP_DIGESTS`=256.

For FRI, opened layers commit one 64-byte leaf per quad; the final layer is sent in the clear and its cap is recomputed from `final_poly`. For AIR, a 64-byte trace leaf contains eight joint rows ($A\|B$, 8 bytes per row). The verifier checks the row at (index&7) $\cdot$ 8 and authenticates the whole leaf at index/8.

5 Fiat–Shamir transcript and proof of work

Framing. `absorb(label,data)` appends

`u64_le(label.len())`‖label‖`u64_le(data.len())`‖data.

The implementation stores the history as a streaming BLAKE3 state. A squeeze clones the state, appends

`u64_le(label.len())`‖label‖`u64_le(counter)`,

finalizes, increments the counter, and absorbs the 32-byte output back into the history. This is byte-identical to hashing the full concatenation but independent of history length.

Challenges. `challenge_f64` uses the first 8 bytes; `challenge_f128` uses 16 bytes; `challenge_bb` reduces an 8-byte word modulo p (bias below 2^{-31}); `challenge_ext` reduces four 4-byte coordinates; `challenge_index` masks low bits for a nonzero power-of-two domain and returns `None` otherwise.

Proof of work. `grind(label,d)` computes

$$\text{head} = H(\text{history}) \| \texttt{u64_le(label.len())} \| \text{label}$$

and searches for the smallest nonce with at least d leading zero bits under $H(\text{head} \| \texttt{u64_le(nonce)})$. The banded parallel search returns the same nonce as a sequential scan. Protocol parameters cap $d \leq 32$; `grind` fails closed above 32 by absorbing nonce 0 and returning 0. `verify_grind` re-hashes and accepts only if the nonce meets the target; it hard-rejects $d > 256$. Queries are sampled after the nonce is absorbed.

Label schedule.

Protocol	Labels, in order
FRI	`fri-dims`, `fri-layer-root` per cap digest, `fri-fold` ×3 per layer, `fri-final`, `fri-pow`, `fri-query` per query
AIR / range / table	`<proto>/params`, `<proto>/column`, `<proto>/alpha` (fixed count)
fold (IVC)	`fold/params`; per step `fold/step`, `fold/trace`, six `fold/alpha`; then one `fold/gamma` per step
lookup	`lookup/witness`, `lookup/table`, `lookup/beta-challenge`, `lookup/beta`, `lookup/m`, `lookup/qw`, `lookup/qt`, `lookup/sums`, `lookup/ctx`, `lookup/r1`, `lookup/r2`
lookup_pcs	`lookup-pcs/commitments`, `lookup-pcs/beta`, `lookup-pcs/quotients`, `lookup-pcs/sum-w`, `lookup-pcs/ctx`, `lookup-pcs/r1`, `lookup-pcs/gamma`, `lookup-pcs/r2`, and one `lookup-pcs/open` label per BaseFold opening
vm	`vm/params`, `vm/trace`, four `vm/alpha`
recursion	`rec/params`, `rec/trace`, seven `rec/alpha`
sumcheck	`sumcheck/claim`; per round `sumcheck/round`, `sumcheck/challenge`

A caller MUST start the transcript with a unique application label. The constructor absorbs `("domain",label)` first.

6 FRI polynomial commitment

6.1 Parameters and commitment

A FRI parameter set is

$$(\beta, Q, F, g, h), \qquad \beta \geq 2, \quad Q \geq 1, \quad 2 \leq F \leq 1024, \quad g \leq 32, \quad h \leq 8,$$

with $F < M$ and M/F a power of four. `commit` maps n coefficients to $M = n\beta$ coset evaluations with a shifted NTT. `commit_evals` accepts already-expanded LDE values with a declared degree bound $d < M$. Both paths validate parameters before the encoding NTT, reject empty or non-power-of-two lengths, enforce $M \leq 2^{27}$, and bind (M, n, d) under `fri-dims` before any challenge. The rate gate

$$d + 1 \leq M/2$$

is enforced at prove and verify.

6.2 Arity-4 folding

Layers are bit-reversed. Each adjacent quad sits at $(x, -x, j_0 x, -j_0 x)$ for the fixed fourth root j_0, sharing x^4. Writing $P(X) = A + BX + CX^2 + DX^3$ in $Y = X^4$, the fold is

$$\text{fold}(e_0, e_1, e_2, e_3) = A + r_0 B + r_1 C + r_2 D,$$

with

$$\begin{aligned} A &= (e_0 + e_1 + e_2 + e_3)/4, \\ B &= ((e_0 - e_1) - j_0(e_2 - e_3))/(4x), \\ C &= ((e_0 + e_1) - (e_2 + e_3))/(4x^2), \\ D &= ((e_0 - e_1) + j_0(e_2 - e_3))/(4x^3), \end{aligned}$$

and $(r_0, r_1, r_2) \in \mathbb{F}_{p^4}^3$. Point inverses are base-field values and are batch-inverted once per layer. Each fold quarters the degree.

6.3 Proof and verification

A proof carries the final polynomial in the clear, one opening per query (four $\mathbb{F}_{p^4}$ values per folded quad plus the middle Merkle path segment), and a PoW nonce. Verification performs:

1. validate dimensions, cap shapes, fold count, and the rate gate;
2. absorb dimensions, cap digests, and the final polynomial;
3. check the PoW nonce against the replayed history;
4. verify the final layer by divided differences against residual $d \gg 2f$ at the true folded domain points;
5. recompute the final layer cap from `final_poly` and compare it to the commitment;
6. sample query indices after grinding and check every Merkle path and fold relation, chaining intermediate parents forward and the last parent into `final_poly`.

The final degree check is non-vacuous: a one-point final layer would let any value interpolate a degree-0 polynomial, so $F \geq 2$ is mandatory and

$$(d \gg 2f) + 1 < F$$

is enforced. A fold-free instance ($F = M$) is rejected because it has no query openings binding `final_poly` to the commitment.

6.4 Conjectured soundness

Under the Johnson-radius heuristic, a word at distance δ from the code survives one query with probability at most $1 - \delta$, where $\delta \approx 1 - \sqrt{\rho}$.

Shape	Rate ρ	Queries Q	Grinding	Conjectured query+PoW
Bench	1/8	12	0	≈ 18 bits
Production	1/16	40	20	≈ 100 bits $(80 + 20)$

Table 4: Conjectured FRI query/PoW schedule. Algebraic error is separate.

These counts exclude finite-field algebraic error. FRI folds, STARK compositions, UPCs/Gemini points and betas, sumcheck rounds, and lookup identities use $\mathbb{F}_{p^4}$, so a degree-d identity collides with probability about d/p^4 per challenge (≈ 124 bits). This remains a query/PoW formula, not an end-to-end theorem.

7 Multilinear polynomials and sumcheck

A multilinear polynomial with n variables stores 2^n evaluations in MSB-first order. `evaluate` folds the least-significant variable first. The equality polynomial is

$$\mathrm{eq}(r, x) = \prod_i (r_i x_i + (1 - r_i)(1 - x_i)).$$

A summand is a sum of terms $f(x) = \sum_k c_k \prod_{j \in \mathrm{factors}_k} \mathrm{mle}_j(x)$ with $c_k \in \mathbb{F}_{p^4}$. Sumcheck proves $S = \sum_{x \in \{0,1\}^n} f(x)$ by sending round polynomials of degree at most the maximum factor count. Verification checks $g_i(0) + g_i(1) = g_{i-1}(r_{i-1})$, samples $r_i \in \mathbb{F}_{p^4}$, and finally checks the oracle value. If the oracle is committed, each final value is opened at the challenge point.

8 Univariate, Gemini, and BaseFold commitments

8.1 UPCs

`upcs` commits a univariate polynomial as a Merkle-committed Reed–Solomon LDE. To prove $g(\beta) = y$, the prover commits $q = (g - y)/(X - \beta)$ and proves both g and q low-degree with FRI; the verifier checks $(X - \beta)q = g - y$ at the quotient's query points. Multi-point openings use I interpolating the claims and $Z = \prod_j (X - p_j)$, with $q = (g - I)/Z$ and $g = I + Zq$. When the points determine the polynomial ($k = \mathrm{poly_len}$), the quotient is zero and the verifier checks g against the interpolant directly. Optional Merkle caps shorten paths and are authenticated against the commitment root.

8.2 Gemini

Gemini commits the monomial-coefficient encoding $g_0(X) = \sum_b c_b X^{\mathrm{index}(b)}$ and proves $f(r) = y$ by folding one variable at a time:

$$g_{i-1}(X) = A_i(X^2) + XB_i(X^2), \qquad g_i(Y) = A_i(Y) + r_i B_i(Y).$$

Each committed layer is opened once with a multi-point UPCs proof at all points its relations use. Layer commitments are absorbed before the fold betas. Proof size is linear, not constant. Each fold identity has degree below 2^{n-i+1}, so one random check misses a nonzero identity with probability at most $2^{n-i+1}/p^4$.

8.3 BaseFold prototype

BaseFold commits the coefficient encoding with FRI and reduces $f(r) = y$ to a coefficient-oracle evaluation $c(\rho)$ with sumcheck. The prover folds the LDE with the Möbius/eq rule

$$(1 - \rho_i)E(X) + \rho_i O(X)$$

rather than the coefficient-encoding rule $E(X) + \rho_i O(X)$, so the final FRI constant is exactly $c(\rho)$. The verifier recomputes

$$w(\rho) = \prod_i (1 - \rho_i + \rho_i r_i)$$

and checks $f(r) = c(\rho)w(\rho)$. Proofs carry one root, $n - 1$ layer roots, sumcheck rounds, and one pair plus one Merkle path per layer per query. Optional caps store a digest frontier per layer and hash it back to the layer root. Batched multi-point openings use the multilinear basis table and are capped at 1024 points. The prototype is not wired into the AIR/STARK stack and has no end-to-end soundness proof; it backs the committed lookup.

9 Lookup arguments

9.1 LogUp in clear

For witness w and table t padded to powers of two, the prover computes first-index multiplicities m, samples β avoiding poles, and forms

$$q_i^w = \frac{1}{\beta + w_i}, \qquad q_j^t = \frac{m_j}{\beta + t_j}.$$

Four sumchecks establish the rational identity $\sum_i q_i^w = \sum_j q_j^t$ and the read-checks $q_i^w(\beta + w_i) = 1$, $q_j^t(\beta + t_j) = m_j$. The verifier has witness and table in clear and recomputes canonical multiplicities exactly; fractional multiplicities are rejected. The API is sound for membership but not hiding.

9.2 Committed LogUp

`lookup_pcs` commits witness, table, multiplicities, and both quotient columns with BaseFold. The verifier sees five commitments and eight openings. The read-checks use the committed columns directly:

$$\mathrm{eq} \cdot \big(q \cdot (\beta + v) - \mathrm{target}\big) = 0,$$

expanded as $\mathrm{eq}qv + \beta\mathrm{eq}q - \mathrm{eq}\,\mathrm{target}$. A fresh γ folds the total multiplicity weight into the sum-t claim:

$$\sum (q^t + \gamma m) = \sum q^w + \gamma W.$$

The verifier rejects $\gamma = 0$. Every final sumcheck value is opened against its commitment. At 1,024-in-1,024 with $B16Q40G20C4$, the prototype measures 273.01 ms prove, 5.77 ms verify, and 642,443 bytes. It is committed-input but not zero-knowledge and does not constrain columns to the base field.

10 AIR systems

10.1 Fibonacci AIR

Trace $A[0] = a_0$, $B[0] = b_0$, $A[i+1] = B[i]$, $B[i+1] = A[i] + B[i]$, claim $B[N-1] = b_{\text{last}}$. With $w_{\text{last}} = \omega^{N-1}$ and $Z_{\text{trans}}(x) = (x^N - 1)/(x - w_{\text{last}})$:

$$\begin{aligned} C(x) = \alpha_0 \frac{a' - b}{Z_{\text{trans}}(x)} + \alpha_1 \frac{b' - (a+b)}{Z_{\text{trans}}(x)} \\ + \alpha_2 \frac{a - a_0}{x - 1} + \alpha_3 \frac{b - b_0}{x - 1} + \alpha_4 \frac{b - b_{\text{last}}}{x - w_{\text{last}}}. \end{aligned}$$

Five `fib/alpha` challenges; $\deg C < N$ iff the trace is valid. Dividing, not multiplying, by Z_{trans} is load-bearing. Blowup is at least 4. Next-row values are the same committed columns at the ω-shifted LDE index, authenticated by a second Merkle path.

10.2 Range check AIR

Value column V plus n bit columns ($1 \le n \le 24$), with challenges α_{decomp}, $\{\alpha_j\}$, α_{bound}:

$$C = \alpha_{\text{decomp}} \frac{V - \sum_j 2^j C_j}{Z_H} + \sum_j \alpha_j \frac{C_j(C_j - 1)}{Z_H} + \alpha_{\text{bound}} \frac{V - \text{value}}{x - 1}.$$

Booleanity doubles the masked degree; masked callers should size blowup and queries for a rate near $2/\beta$. The verifier caps the declared bound at the largest honest masked degree.

10.3 Table membership AIR

$$C(x) = \alpha_{\text{member}} \frac{\prod_{t \in T}(v - t)}{Z_H(x)} + \alpha_{\text{bound}} \frac{v - \text{value}}{x - 1}.$$

The honest composition bound is $|T|(n + k - 1) - n$ for n rows and k maskers. The tiny-table `for_security` tuner floors blowup at 4 so FRI always has a valid final layer.

10.4 Hash-folded IVC

Segments of `seg_steps` Fibonacci rows are committed independently with explicit in/out claims. The verifier checks chain links, then verifies one shared FRI proof over

$$V = \sum_s \gamma_s C_s$$

with per-step challenges. A far-from-code segment keeps the combination far with overwhelming probability over the gammas. The statement is existential over the number of segments; callers requiring an exact step count must check `proof.steps.len()`. The accumulator retains each segment's LDE, Merkle tree, and composition until finalization.

10.5 Register-machine execution

A public two-register straight-line program runs from a private initial state to a public output. Opcodes are `Add`, `Mul`, `Swap`, `SetA`, `SetB`, and `Halt`; the last row is the public output and has no outgoing transition. Public one-hot selectors and immediate polynomials drive transition numerators T, U:

$$C(x) = \alpha_0 \frac{T(x)}{Z_{\text{trans}}(x)} + \alpha_1 \frac{U(x)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{A(x) - a_{\text{last}}}{x - w_{\text{last}}} + \alpha_3 \frac{B(x) - b_{\text{last}}}{x - w_{\text{last}}}.$$

Plain composition degree is below $2n - 1$, so blowup is at least 8. Masked columns have degree padded $- 1$; the `Mul` transition is quadratic, so the prover declares $2(n + n_{\text{rand}} - 1)$ and the verifier accepts up to $2 \cdot \text{padded} - 2$. Each query reveals the current and shifted rows, so the mask budget is $2Q$. The program and output remain public; there is no control flow or memory.

10.6 Recursion fragment

`recursion` proves one FRI query's fold chain as a small STARK. Public inputs are the outer `lde_len`, leaf, per-depth challenges, the layer-0 quad, and the output; the witness is the sequence of quads and folded outputs. Constraints cover the fold identity, the selected next-layer element, the initial quad, and the final output. The fragment does not yet verify BLAKE3 paths, the final degree check, or transcript derivation inside the AIR.

11 Zero-knowledge masking

Masked proving maps a column T of degree $< n$ to

$$M(x) = T(x) + Z_H(x)R(x),$$

where R has n_{rand} uniform coefficients. Z_H vanishes on every base row, so trace values and AIR quotient identities are preserved. Off-domain, the mask term is nonzero. Evaluations of R at k distinct points are jointly uniform when $n_{\text{rand}} \geq k$ (Vandermonde), giving perfect hiding of query responses.

Protocol	Revealed values per column per query	Minimum n_{rand}
`range`, `table`	one point	Q
`vm`	current and shifted rows	$2Q$
`air`	eight-row leaf and shifted leaf	$16Q$

Table 5: Masking budgets for the values actually revealed.

The composition degree grows accordingly; callers must size `final_size` for the masked LDE. Verification is unchanged, and masked false statements still fail. This is query-response hiding, not full program privacy: public inputs, Merkle roots, and composition commitments remain visible. `lookup`, `sumcheck`, `upcs`, `gemini`, and `basefold` are not hiding.

12 Wire format

All proofs use a canonical versioned encoding, `VERSION=6`: little-endian integers, `u32` counts, canonical base and extension fields, 32-byte digests, and 0/1 direction bytes. The decoder rejects trailing bytes, truncation, non-canonical fields, and oversized counts before allocation.

Limit	Value
`MAX_QUERIES`	$2^{20} = 1{,}048{,}576$
`MAX_FOLDS`	64
`MAX_PATH_STEPS`	128
`MAX_CAP_DIGESTS`	256
`MAX_ELEMS`	$2^{27} = 134{,}217{,}728$
`MAX_STEPS` (IVC)	$2^{20} = 1{,}048{,}576$
FRI `MAX_FINAL_SIZE`	1024
maximum Merkle cap height	8
BaseFold `MAX_BATCH`	1024
BaseFold `MAX_VARS`	16
Gemini `MAX_VARS`	20
UPCs `MAX_POINTS`	16
VM `MAX_PROGRAM`	2^{20}
recursion `MAX_FOLDS`	16

Table 6: Hard decoder and protocol limits.

Proof bodies:

- FRI: final polynomial, PoW nonce, query count, per-query fold witnesses (four $\mathbb{F}_{p^4}$ values plus path).
- AIR/range/table/vm/recursion: column cap, FRI commitment and proof, opening count, per-opening index/values/paths (and 64-byte leaves for AIR).
- fold: step count, per-step claims/cap/openings, shared FRI commitment and proof.
- BaseFold: sumcheck sum and rounds, layer roots, caps, PoW nonce, queries.
- UPCs: two FRI instances, quotient commitment/proof, openings.
- Gemini: layer commitments, values, nested UPCs openings.
- lookup: beta, multiplicities, quotient columns, sums, four nested sumcheck proofs.
- lookup_pcs: beta, sum, four nested sumcheck proofs, eight BaseFold openings.
- sumcheck: claimed sum, rounds, final values.

13 Parameter validation and sizing

Every entry point validates:

- domains are nonempty powers of two within 2^{27};
- blowup is a power of two (≥ 4 for AIR/fold, ≥ 2 for FRI/range/table, ≥ 8 for Gemini/VM);
- queries ≥ 1 and $\leq 2^{20}$;
- `final_size` ≥ 2, power of two, ≤ 1024, and strictly below the LDE size;
- M/F is a power of four;
- cap height ≤ 8;
- grinding bits ≤ 32;
- rate gate $d + 1 \leq M/2$;
- residual bound non-vacuous: $(d \gg 2f) + 1 < F$;
- recursion outer `lde_len` $\leq 2^{27}$.

`Params::for_security(len,...,target)` selects blowup 16, computes millibits per query from the rate and composition degree, splits the target between queries and grinding, and chooses `final_size` and `cap_height` with `tune_shape`. For target 100, the AIR/FRI production shape is rate 1/16, 40 queries, and 20-bit grinding. The FRI tuner never selects a one-point final layer and caps normal finals at 32; the hard verifier cap is 1024.

Protocol	Degree bound	Mask budget
AIR	$N + n_{\text{rand}} - 1$	$16Q$
Range	$N + 2n_{\text{rand}} - 2$	Q
Table	$\lvert T\rvert(N + n_{\text{rand}} - 1) - N$	Q
VM	$2(N + n_{\text{rand}} - 1)$	$2Q$

Table 7: Masked composition bounds and mask budgets.

14 Security analysis

Assumptions. BLAKE3 collision/preimage resistance; FRI soundness under the Johnson-radius heuristic; Fiat–Shamir in the (quantum) random-oracle model; standard Reed–Solomon list-decoding bounds. No discrete-log, pairing, or setup assumptions.

Adversary model. The adversary controls proof bytes, commitments, public parameters, and public statements. It may choose malformed counts, paths, caps, field encodings, and dimensions. The verifier must fail closed, never panic, and never allocate based on an unchecked count. The adversary may also choose the prover's witness and strategy; soundness is argued against a malicious prover, not only random errors.

Enforced invariants.

1. Rate gate at prove and verify.
2. Verifier-derived domains and query indices.
3. Non-vacuous final divided-difference check.
4. Cap/path shape pins before hashing.
5. Direction bits bound to claimed indices.
6. `fri-dims` dimension binding.
7. Canonical-only decoding with hard caps.
8. PoW bound to the full history; queries sampled after grinding.
9. Committed read-checks in `lookup_pcs`.
10. Correct masked composition bounds in VM.
11. Next-row openings authenticated against the same trace cap.
12. No allocation before count limits are checked.

Closed findings. Internal adversarial review has closed Merkle path index substitution across all verifiers and hand-rolled FRI/BaseFold walks; one-point FRI final layers and fold-free FRI; final cap not bound to `final_poly`; `lookup_pcs` read-checks using uncommitted shifted MLEs; VM masked `Mul` degree under-declaration; AIR/VM ZK mask budgets undercounting revealed values; batched BaseFold openings using the equality table instead of the multilinear basis table; and prover allocation ordering, cap-height/roundtrip limits, recursion domain bounds, checked buffer arithmetic, and transcript/grinding hardening.

Residual risk. The system is unaudited. The query/PoW figures are conjectured. The small-table AIR scales with table size. `lookup_pcs` is not zero-knowledge and does not constrain columns to the base field. BaseFold, Gemini, VM, and recursion are prototypes. Side channels in the host OS, compiler, or CPU are out of scope. Do not deploy for high-value statements before an external audit.

15 Evidence and reproducibility

Suite	Tests	Evidence
Library	277	KATs, Horner/`%` cross-checks, parallel determinism, protocol roundtrips
`security.rs`	13	479 hostile attempts, zero accepted
`redteam.rs`	10	37 targeted + 500 garbage, zero accepted/panics
`hardening.rs`	11	Hostile params, exhaustive per-bit AIR decoder fuzz, 14,000 mutations, zero panics
Doc tests	5	Public examples
Total	**316**	Plus coverage-guided `fuzz/decode`

Table 8: Validation matrix on the current tree.

Workload (production query/PoW)	Prove	Verify	Size
AIR 1,024 steps	37.05 ms	0.48 ms	68,573 B
AIR 8,192 steps	49.81 ms	0.59 ms	101,233 B
AIR 65,536 steps	115.49 ms	0.90 ms	141,641 B
AIR **1,048,576** steps	**1,733.55 ms**	**1.13 ms**	**203,553 B**
FRI poly-1,024	21.50 ms	0.31 ms	43,174 B

Table 9: Production AIR/FRI measurements, 2026-11-05.

The 1M-step row is 1.653 μs/step and 604,872 steps/s. The full gate is `python3 tools/check.py`; it runs fmt, Markdown links, rustdoc, pedantic clippy, debug tests, release tests, and the benchmark regression. The baseline pins all 24 rows: proof sizes byte-for-byte, times within 3× of `tools/baseline.json`.

A Error taxonomy

Error	Meaning
`EmptyInput`	Empty table/vector where at least one element is required
`IndexOutOfBounds`	Index outside a committed domain
`InvalidDomainSize`	Non-power-of-two, zero, or above the domain ceiling
`InvalidBlowup`	Blowup below the protocol floor or not a power of two
`NoQueries`	Zero FRI queries
`BadMerklePath`	Failed path, cap, shape, or index check
`FriVerificationFailed`	Fold, degree, PoW, or identity failure
`MalformedInput`	Canonical encoding or parameter violation
`RngUnavailable`	OS RNG failure

B Public API map

Module	Main entry points
`babybear`	`Field`, `batch_invert`
`ext4`	`Ext4`, `from_base`, `mul_base`, `inv`
`gf2`	`F8`, `F64`, `F128`, `splitmix64`
`hash`	`blake3_256`, `Hasher`
`merkle`	`MerkleTree`, `hash_leaf`, `hash_node`, `capped_shape`
`transcript`	`Transcript`, `challenge_*`, `grind`, `verify_grind`
`ntt`	`forward`, `inverse`, `interpolate`, `evaluate_on_coset`, `evaluate`
`fri`	`Params`, `commit`, `commit_evals`, `open`, `verify`
`mle`	`Mle`, `eq_evals`, `evaluate`, `fix_last`
`sumcheck`	`Term`, `Proof`, `prove`, `verify`, `interpolate_at`
`upcs`	`commit`, `open`, `open_multi`, `verify`, `verify_multi`
`gemini`	`commit`, `commit_evals`, `open`, `verify`
`basefold`	`commit`, `open`, `open_multi`, `verify`, `verify_multi`
`lookup`	`prove`, `verify`
`lookup_pcs`	`prove`, `verify`, `Commitments`
`air`	`Params`, `build_trace`, `prove`, `prove_zk`, `verify`

Module	Main entry points
`range`	`Params`, `build_columns`, `prove`, `prove_zk`, `verify`
`table`	`Params`, `prove`, `prove_zk`, `verify`
`fold`	`Params`, `Accumulator`, `verify`
`vm`	`Params`, `Op`, `prove`, `prove_zk`, `verify`
`recursion`	`Params`, `prove`, `verify`
`blind`	`mask_coeffs`, `masked_column_degree`
`rng`	`ChaCha20`, `os_seed`, `fresh_stream`

C Parameter quick reference

Constructor	Key validation
`fri::Params::new`	blowup ≥ 2, queries ≥ 1, final $2 \dots 1024$
`air::Params::new`	N power of two ≥ 2, blowup ≥ 4, $M \leq 2^{27}$
`range::Params::new`	bits $1 \dots 24$, blowup ≥ 2
`table::Params::new`	table checked at prove/verify, blowup ≥ 2
`fold::Params::new`	segment ≥ 2, blowup ≥ 4
`vm::Params::new`	blowup ≥ 8, program $\leq N - 1$
`recursion::Params::new`	folds $1 \dots 16$, outer LDE $\leq 2^{27}$
`basefold::Params::new`	blowup ≥ 2, cap ≤ 8, vars ≤ 16
`gemini::Params::new`	blowup ≥ 8, vars ≤ 20
`upcs`	points ≤ 16, blowup ≥ 2, LDE $\leq 2^{27}$

D Glossary

Term	Meaning
AIR	Algebraic Intermediate Representation
FRI	Fast Reed–Solomon IOP of Proximity
LDE	Low-degree extension
MLE	Multilinear extension
PCS	Polynomial commitment scheme
PoW	Proof of work / Fiat–Shamir grinding
STARK	Scalable Transparent Argument of Knowledge
UPCs	Univariate polynomial commitment scheme
ZK	Zero knowledge

E Reproducibility checklist

1. `cargo fmt -check`
2. `python3 tools/check_links.py`
3. `RUSTDOCFLAGS="-D warnings" cargo doc -no-deps`
4. `cargo clippy -all-targets - -D warnings`
5. `cargo test` and `cargo test -release`

6. `cargo bench` (24 rows)
7. `python3 tools/check.py` (full gate)